In [ ]:
```
# Time Series Python complete program to predict customers without functions
## Date 3.10 V2
```

In [ ]:
```
import pandas as pd
import holidays
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from xgboost import XGBRegressor
df = pd.read_csv("Datasets/customers_data.csv")
df['time'] = pd.to_datetime(df['time'], errors='coerce') # parse with offsets
df = df.set_index('time')
df = df.dropna()
df['year'] = df.index.year
df['month'] = df.index.month
df['day'] = df.index.day
df['day_of_week'] = df.index.dayofweek
df = df.copy()
df["previous_day_customers"] = df["customers"].shift(1).fillna(0)
df = df.copy()
    # rolling mean up to yesterday (no leakage)
df["rolling_mean"] = df["customers"].shift(1).rolling(window=5, min_periods=1).mean()
df["rolling_mean"] = df["rolling_mean"].fillna(df["rolling_mean"].mean())
df = df.copy()
df["exponential_smoothing"] = df["customers"].shift(1).ewm(span=5, adjust=False).mean()
df["exponential_smoothing"] = df["exponential_smoothing"].fillna(df["exponential_smoothing"].mean())
df = df.copy()
roll_autocorr = (
df["customers"].shift(1) # avoid peeking at today
.rolling(window=14, min_periods=max(1 + 1, 3))
.apply(lambda x: x.autocorr(lag=1), raw=False)
)
df["autocorrelation"] = roll_autocorr.fillna(0) # safer default than mean
df = df.copy()
if not isinstance(df.index, pd.DatetimeIndex):
    raise ValueError("DataFrame index must be a DatetimeIndex.")
fi_holidays = holidays.Finland(years=range(2019, 2025))
holiday_idx = pd.DatetimeIndex(pd.to_datetime(list(fi_holidays.keys())))
d = df.index.normalize()
```

```
df["is_holiday"] = d.isin(holiday_idx).astype(int)
df["day_before_holiday"] = (d + pd.Timedelta(days=1)).isin(holiday_idx).astype(int)
df["day_after_holiday"] = (d - pd.Timedelta(days=1)).isin(holiday_idx).astype(int)
df = df.dropna()
target = "customers"
features = [col for col in df.columns if col != target]
X = df[features]
y = df[target]
# === Train/Test Split (time-aware, no shuffle) ===
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, shuffle=False
)
# === Model ===
model = XGBRegressor(
n_estimators=300,
learning_rate=0.1,
max_depth=5,
subsample=0.8,
colsample_bytree=0.8,
random_state=42
)
# === Train ===
model.fit(X_train, y_train)
# === Predict ===
y_pred = model.predict(X_test)
# === Evaluate ===
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"MAE: {mae:.2f}")
print(f"R²: {r2:.2f}")
```

In [ ]: